

SOUGATA SAHA

IT-A2-037

OOS ASSIGNMENT 5

Q1. Write a java program to implement Factory pattern..

Ans: interface Shape {

void draw();

}

class Circle implements Shape {

public void draw() {

System.out.println("Circle is Drawn");

}

}

class Square implements Shape {

public void draw() {

System.out.println("Square is Drawn");

}

}

class Rectangle implements Shape {

public void draw() {

System.out.println("Rectangle is Drawn");

}

}

abstract class ShapeFactory {

abstract Shape getShape();

}

class GetCircle extends ShapeFactory {

public Shape getShape() {

return new Circle();

}

}

class GetSquare extends ShapeFactory {

public Shape getShape() {

return new Square();

}

```

}

class GetRectangle extends ShapeFactory {
    public Shape getShape() {
        return new Rectangle();
    }
}

class FactoryPatternDemo {
    public static void main(String[] args) {
        ShapeFactory shapeFactory = new GetCircle();
        Shape shape = shapeFactory.getShape();
        shape.draw();
        shapeFactory = new GetSquare();
        shape = shapeFactory.getShape();
        shape.draw();
        shapeFactory = new GetRectangle();
        shape = shapeFactory.getShape();
        shape.draw();
    }
}

```

Output:

```

[be2337@localhost Assignment4]$ javac ql.java
[be2337@localhost Assignment4]$ java FactoryPatternDemo
Circle is Drawn
Square is Drawn
Rectangle is Drawn
[be2337@localhost Assignment4]$ █

```

Q2. Write a java program to implement decorator pattern.

Ans:

```

interface Shape {
    void draw();
}

class Circle implements Shape {

```

```
        public void draw() {
            System.out.println("Drawing Circle");
        }
    }

    class Rectangle implements Shape {
        public void draw() {
            System.out.println("Drawing Rectangle");
        }
    }

    class ShapeDecorator implements Shape {
        public Shape shape;
        public ShapeDecorator(Shape shape) {
            this.shape = shape;
        }
        public void draw() {
            this.shape.draw();
        }
    }

    class RedShapeDecorator extends ShapeDecorator {
        RedShapeDecorator(Shape shape) {
            super(shape);
        }
        private void setRedBorder() {
            System.out.println("Adding RedBorder");
        }
        public void draw() {
            super.draw();
            this.setRedBorder();
        }
    }

    class DecoratorPatternDemo {
        public static void main(String[] args) {
```

```

        Circle circle = new Circle();

        Rectangle rectangle = new Rectangle();

        circle.draw();

        rectangle.draw();

        RedShapeDecorator circleDecorator = new RedShapeDecorator((Shape) circle);

        RedShapeDecorator rectangleDecorator = new RedShapeDecorator((Shape)
            rectangle);

        circleDecorator.draw();

        rectangleDecorator.draw();

    }
}

```

Output:

```

[be2337@localhost Assignment4]$ javac q2.java
[be2337@localhost Assignment4]$ java DecoratorPatternDemo
Drawing Circle
Drawing Rectangle
Drawing Circle
Adding RedBorder
Drawing Rectangle
Adding RedBorder
[be2337@localhost Assignment4]$ █

```

Q3. Write a java program to design mediator pattern.

Ans:

```

class User {

    private String name;

    User(String name) {

        this.name = name;

    }

    public String getName() {

        return this.name;

    }

    public void setName(String name) {

        this.name = name;

    }

}

```

```

        public void sendMessage(String message) {
            System.out.println("@ " + name + " : " + message);
        }
    }

    interface Mediator {
        void showMessage(User user, String msg);
    }

    class ChatRoom implements Mediator {
        public void showMessage(User user, String msg) {
            user.sendMessage(msg);
        }
    }

    class MediatorPatternDemo {
        public static void main(String[] args)
        {
            User user1 = new User("Soham");
            User user2 = new User("Afham");
            User user3 = new User("Sayan");
            Mediator chatRoom = new ChatRoom();
            chatRoom.showMessage(user1, "Hi, Nice to meet you all");
            chatRoom.showMessage(user2, "Good Morning Everyone!");
            chatRoom.showMessage(user3, "Hello, Hi I am fine");
        }
    }

```

Output:

```

[be2337@localhost Assignment4]$ javac q3.java
[be2337@localhost Assignment4]$ java MediatorPatternDemo
@Soham : Hi, Nice to meet you all
@Afham : Good Morning Everyone!
@Sayan : Hello, Hi I am fine
[be2337@localhost Assignment4]$ █

```